Szoftvertechnológia és -technikák

11. Előadás - A vízesés és a RUP módszertan

BME AUT
Automatizálási és
Alkalmazott
Informatikai Tanszék

Szoftverfejlesztési módszerek

- Egy mérnöki diszciplína, amely fókuszában a szoftver-alapú rendszerek fejlesztésének aspektusai szerepelnek.

Szoftverfejlesztési módszerek

- Vízesés
- Spirális
- Iteratív és inkrementális
- Agilis

A szerződés: megállapodás

- Mit adok – mit kapok
  - Keretek: idő, minőség, …

Két alap modell

1. Projekt alapú
2. Költség alapú

1. Projekt alapú

- A szerződésben rögzített témák
  - „specifikáció”, feature set
  - Ellenérték
  - Egyéb paraméterek…
- Kinél van a kockázat?
  - A fejlesztőnél:
  - Ha a projekt mérföldköveinél szállítja ami benne van a szerződésben, kifizetik, de addig …
- Van-e mozgástere a megrendelőnek?
  - Nincs: ha a végén elkészül amit kért, ki kell fizetni

2. Költség alapú

- A szerződésben rögzített témák
  - Fejlesztői/… erőforrások időarányos ára
  - Minőségi és egyéb paraméterek
- Kinél van a kockázat?
  - A megrendelőnél
  - Bármi is lesz a projektből, a ráfordított időt ki kell fizetnie
- Van-e mozgástere a megrendelőnek?
  - Igen: bármit kérhet bármikor, úgyis kifizeti

Mi az alapvető probléma?

- Minden szervezet stratégiai célokban és költség keretekben gondolkodik
- Stratégia: jól definiált jövőbeni célok
- A pontos részletekre bontás nehéz
- Az elkészült projekt nem mindig juttatja el a céget a kitűzött stratégiai célhoz
  - Az igazi probléma gyakran a megrendelő oldali folyamatokban, struktúrában stb. rejlik
- **Az informatika kiszolgáló terület! Alkalmazkodnunk kell!**

Egy projekt „megnyerése”

- Szabályozott környezet
  - Például előre rögzített pontszámítási módszer, …
- Szabad környezet
  - Gyakran kevésbé publikált elbírálási módszer
- Mindkét esetben nagy tere van a lobbinak

Leggyakoribb kiválasztási módszer

- Projekt alapú
  - Fix költségvetés
  - Fix feature set
  - Ár alapú döntés

- Kockázat a fejlesztőnél

A szabályrendszerekről

- Minden szabályrendszer motiválja a résztvevőket valamilyen irányba
  - Lehet a szabályalkotó tudatos választása
  - Váratlan „mellékhatások”

- Ezekkel érdemes tisztában lenni
  - Kihasználni
  - A veszélyeket időben felismerni

Mire motivál a projekt alapú szerződés?

- Fejlesztők
  - Feature freeze: semmi változtatás az eredeti specifikáción, merev ellenállás, „bármi áron”
    - Vagy olyan változás, amit a megrendelő kifizet…
- Megrendelők
  - Minél több/gazdagabb funkció fejlesztése
  - Előzetes felmérés: minden belekerüljön amire szükség van/lehet
    - Költség tervezés, megtérülés vizsgálat, …

A projekt alapú megközelítés veszélyei

- Az előzetes specifikáció (ajánlati kiírás) sosem lehet teljes
  - Viszont ennek alapján készül az ajánlat
- A későbbi változtatások nehézkesek

- Okai:
  - A megrendelő nem tudja, nem veszi a fáradtságot
  - Nem az írja a specet mint aki használni fogja
  - Változó gazdasági, technológiai körülmények
  - Az ördög mindig a részletekben rejlik

A kockázatok mérséklése

- Ajánlatba beépített puffer a fejlesztői oldalon
  - Verseny környezetben kockázatos
- Rugalmasság, mindkét oldalon
  - Barter megállapodások: nem módosul a teljes projekt összeg
- Lépcsőzetes kidolgozás és ajánlatok
  - Részletes, kifizetett tervek
  - Hátránya: lassú átfutás, sok döntési pont

Mire motiválhat a költség alapú szerződés?

- Fejlesztők
  - Hosszútávú projekt
  - A megrendelő folyamatosan kapjon értéket
  - Minél több munka, változtatások

- Megrendelők
  - Alul specifikálás: nem kell előre specifikálni, hiszen közben bármikor módosítható a funkció halmaz

A költség alapú megközelítés veszélyei

- Előzetes specifikáció/végig gondolás hiányában rengeteg, nagy költségű változtatás
  - A változtatások gyakran architekturálisak
- Nem készül el a kívánt alap funkcionalitás sem
  - Folyamatos változtatások
- A költségek irreálisan nagyokká válnak
  - Különösen összevetve az elkészült terméket a rá költött összeggel

A kockázatok mérséklése

- Minden agilis módszertan nagy hangsúlyt helyez a tervezésre, közös vízióra!
  - A termék elhelyezése a megrendelő stratégiai koncepciójában, életében

- Előzetes (nem kötelező érvényű) költség becslések

A rugalmasságról - kérdések

- „Miért nem úgy van elkészítve az alkalmazás, hogy ezt egy nap legyen beletenni?”
- „Ez miért nem paraméterezhető, miért van beledrótozva?”
- „Igaz, hogy nem kértük az ajánlatban kifejezetten, de minden minőségi informatikai projektben ezt paraméterezhetően csinálnák meg…”
- „Ez miért nem egy adatbázis mező amit mi tudunk módosítani, miért kell ehhez fejlesztés?”

A rugalmasságról - válaszok

- A rugalmasság sosincs ingyen
  - Ami igen, azt tessék belefejleszteni! ☺
- Előre nem tudni, hol lesz rá szükség!
- Az alkalmazást minden irányú bővítésre felkészíteni tipikus hiba
  - Költségek: fejlesztés és karbantartás
  - Overarchitecting, KISS/Yagni elveknek ellentmond stb.
  - Könnyen inkonzisztenciát okozhat
- Ezért nem feltétlenül hiba, ha valamit nem lehet paraméterezni pedig meg lehetett volna úgy is írni

Projekt alapú megközelítés elvárásai

- Fel lehessen használni a készen lévő „specifikációt”
- Ellenőrizhető, követhető dokumentáció
- Lehessen tervezni
  - Tervező eszközökkel megközelíthető architektúra
  - Esetleg kódgenerálás
  - A terveket az implementációs fázisban ideálisan tudjuk felhasználni, ne legyen kétszeri munka
- Stabil technológiák, eszközök használata
  - Tartani kell a határidőket

Költség alapú megközelítés elvárásai

- „Változtatás tűrés”
- Refaktorálás regresszió nélkül
- A kódban történt változtatások gyors ellenőrzése
- Jól unit tesztelhető alkalmazás
  - A jó unit tesztek jellemzői
- Rugalmas, bővíthető architektúra

Az üzleti modell és a módszertanok

- Az üzleti modell / szerződés típus erősen befolyásolja az alkalmazott módszertant
  - Fix költség – vízesés
  - Költség alapú – evolúciós
- De vannak egyéb – üzleti – tényezők
  - Például: a szállító hosszú távon tartja karban az alkalmazást, jó minőségű, olcsó javításokat, fejlesztéseket szeretne adni hosszú távon -> evolúciós módszertant választ az esetlegesen nagyobb kezdeti költségek ellenére

Összefoglalás

- Minden fejlesztési modellnek megvannak a maga erősségei és gyengeségei
  - A kiválasztás soktényezős
- Néhány évente újabb és újabb fejlesztési módszerek jelennek meg
  - Érdemes követni az új megoldások fejlődését
- Mindig legyünk tisztában a környezettel amiben dolgozunk!

Vízesés modell

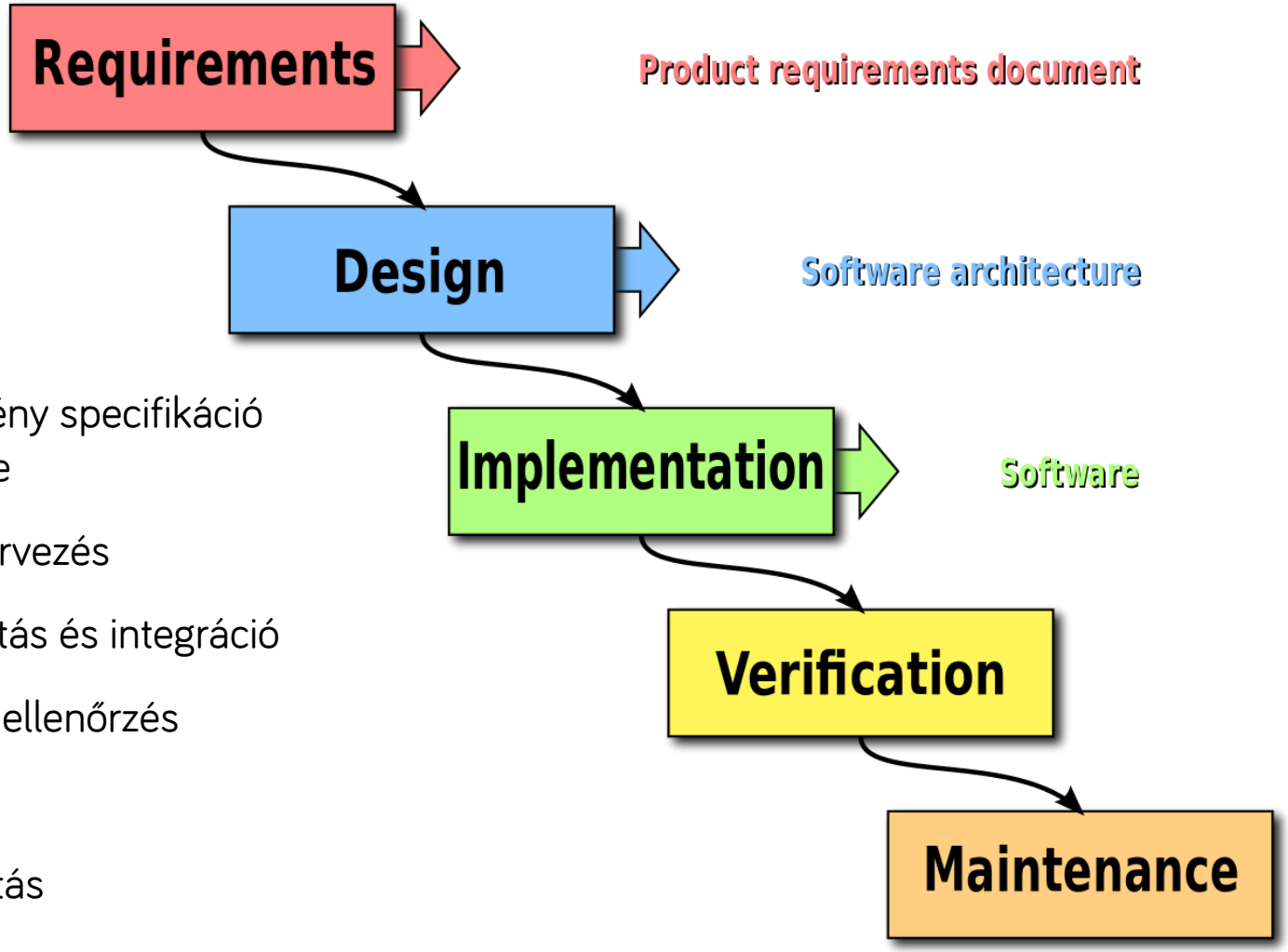
Vízesés modell

- **Példa:** A BKV megbízza a szoftverfejlesztő csapatunkat egy elektronikus jegyrendszer fejlesztésével.
- Jól rögzített határideje van:
  - az átadást 2022-re tűzik ki.
- Alphától omegáig minden ránk van bízva:
  - Mindent nekünk kell kitalálni, megvalósítani, telepíteni, átadni, karbantartani.
- A megrendelővel a projekt elején és végén találkozunk.

Vízesés modell

- Az elmúlt 50 év leggyakrabban használt modellje
- Ma is nagyon gyakori, főleg hazánkban
- Lineáris modell
  - Egymást követik a lépések
  - Akkor kezdünk el egy új fázist, ha az előző befejeződött

Vízesés modell lépései



1. Követelmény specifikáció elkészítése
2. Szoftvertervezés
3. Megvalósítás és integráció
4. Tesztelés, ellenőrzés
5. Telepítés
6. Karbantartás

1. lépés: Követelmények

- Részletes, közös képet alakít ki a Megrendelő és a Szállító között az elkészítendő termékről
- Akkor lépünk tovább, amikor elfogadásra került a specifikáció

1. lépés: Követelmények

- Egy specifikáció elemei:
  - A projekt célja, helye a vállalatnál
  - Terminológia, definíciók
  - Felelősök
  - Felhasználói csoportok
  - Feltételezések, kényszerek, elő-követelmények, függőségek
  - Funkcionális követelmények – Folyamatok, interfészek stb. tételes felsorolása és részletezése
  - Nem funkcionális követelmények – Teljesítmény, biztonság, bővíthetőség, formátum, …
  - Becsült határidők
  - Átadás / átvételi tesztek

2. lépés: Tervezés

- Koncepcionális tervek:
  - Architektúra, felhasznált keretrendszer
  - Interfészek, modul határok
  - Részletes adatbázisterv
  - Folyamatok leírása
  - Képernyőtervek, navigációs térkép, stb.
  - Használati minták
  - Biztonsági, tesztelési tervek
- Prototipizálás:
  - Iterációk a felhasználói visszajelzések alapján

2. lépés: Tervezés

- A tervezés lépése után a további módosítások már költségesek!
- A vízesés modell alapja a jó terv.
  - Megrendelő által validálható tervek – például képernyő képek, folyamat ábrák!
  - Megrendelői oldalon valódi ellenőrzés a munkát végzők által (kulcsfelhasználók)!
  - Nehézséget és kockázatot jelent, ha a megrendelő közben egy transzformációban van benne.

3. lépés: Megvalósítás és Integráció

- Ebben a lépésben indul el a fejlesztés.
- Munka dekomponálása, fejlesztési tervek
- Többnyire a leghosszabb idő a teljes folyamatban

4. lépés: Tesztelés

- A minőség ellenőrzők megvizsgálják a szoftvert, hogy a specifikáció szerint működik-e?
- A hibák visszajutnak a fejlesztőkhöz és a tesztelés elölről indul
- Gyakran használunk átmeneti állapotot: béta verzió, meghívott felhasználók stb.

5. lépés: Telepítés

- Telepítés, terjesztés
- Pontosan ugyanaz a verzió menjen ki, mint amit a tesztelők jóváhagytak!

6. lépés: Karbantartás

- Karbantartási / továbbfejlesztési fázis
  - Hibajavítás, tesztelés, …
- A karbantartás a legdrágább fázis
  - A hibajavítás kiemelten költséges lehet

A vízesés modell előnyei

- A megrendelők és a fejlesztők hamar (a projekt elején) megállapodnak, tisztázzák a fejlesztés fázisait.
- A projekt fejlődése könnyen mérhető és nyomonkövethető
  - Melyik fázisban vagyunk éppen
  - Tudjuk (jó becslésünk van arról, hogy mennyi van hátra

A vízesés modell előnyei

- Nem kell mindenkinek folyamatosan a projekten dolgoznia
  - Pl. A tesztelőknek a tesztelés fázisában kell dolgozniuk, előbb nem
- A megrendelő (folyamatos) jelenléte nem szükséges
- Ha a tervezés jó, akkor előre becsülhető, mikorra lesz kész a fejlesztés és melyik fázis mennyi időt vesz igénybe

A vízesés modell kritikája

- Nem az készül el, amire szükség van végül
  - Gyakran nem kapunk visszajelzéseket a termékről addig, amíg az el nem készült
- A végül megjelenő új követelmények koncepcionális változásokat indukálhatnak, ami újratervezést igényelhet
  - Jelentőssé válhat a kidobott idő, energia
- A tervezők gyakran nincsenek tisztában az implementációs részletekkel
  - Néha egyszerű tervváltoztatás helyett bonyolult implementáció készül el

A vízesés modell kritikája

- A tervező eszközök nem integrálódnak jól az implementációt segítő eszközökkel
- Túl hosszú lehet a teljes fejlesztési fázis, nincs részfelhasználás

Mikor használjuk?

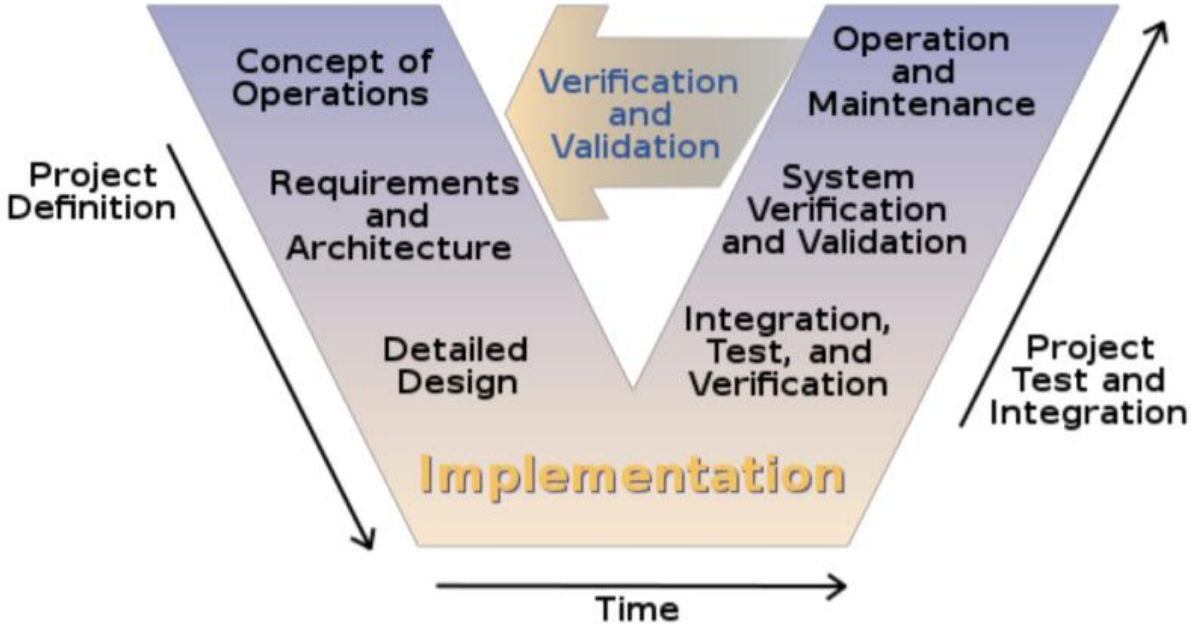
- Fix projekt alapú a megállapodásunk
- A megrendelő csak a mérföldköveknél van jelen
- Pontos határidővel dolgozunk
  - Például törvényi előírások
- Nem változnak a követelmények útközben

Esettanulmány (olvasmány)

- https://medium.com/@kawomersley/would-you-rather-be-late-or-wrong-a-strategic-case-for-waterfall-development-35d81911022c

V-modell

- Fejlesztési módszertan, a vízesés modell kiterjesztése
- Megvan a kapcsolat a fejlesztési és a tesztelési lépések között

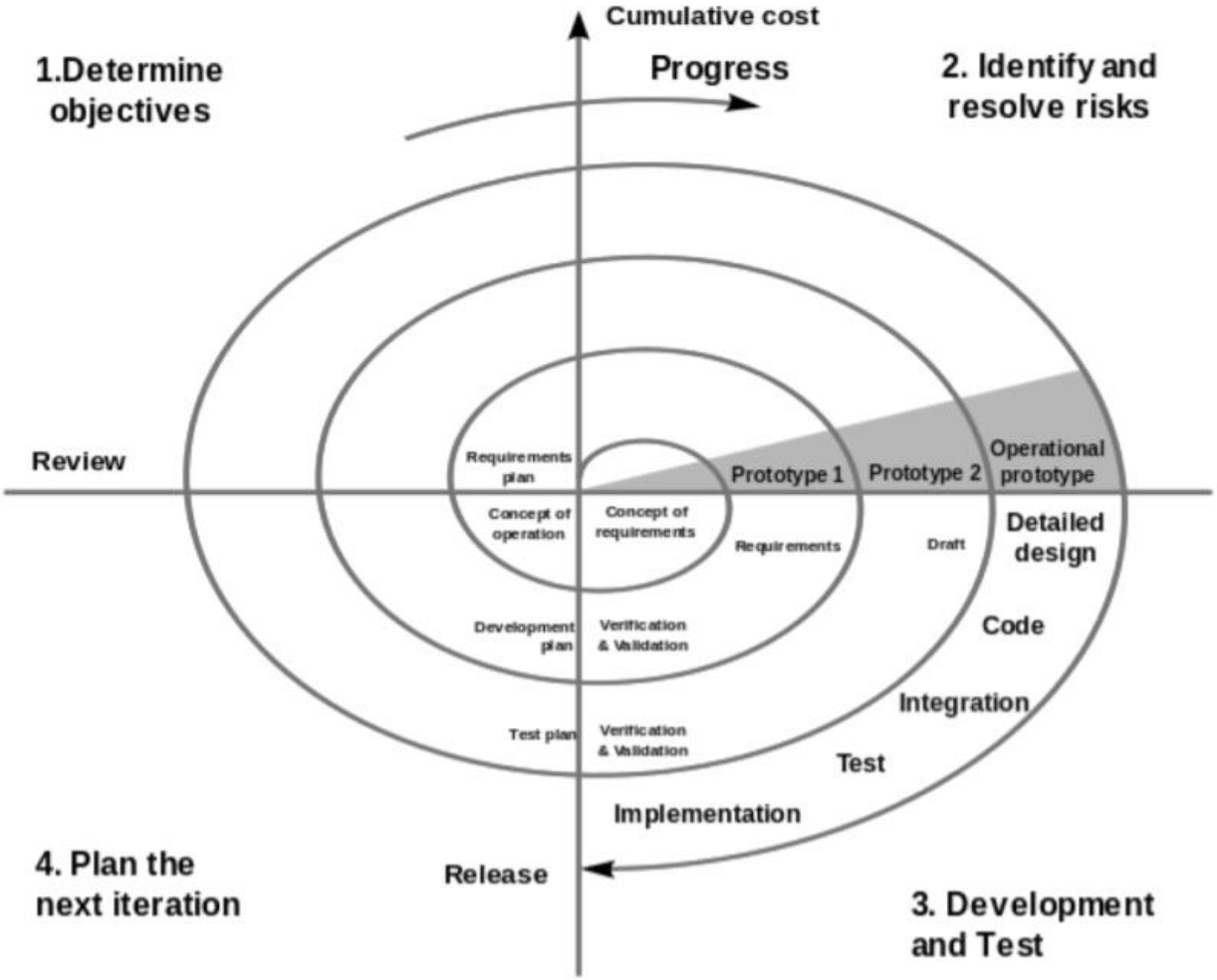


Evolúciós modellek

- A vízesés modell problémáinak elkerülése érdekében született meg egy másfajta megközelítés
- Evolúciós megközelítés
  - Több ciklus, mindegyik a termék egy részéért felel
  - Ez lehetővé teszi a követelmények módosítását

Jelenleg az agilis megoldások a legnépszerűbbek az evolúciós megközelítések közül

Spiral


Cumulative cost
Progress
1.Determine objectives
2. Identify and resolve risks
Review
Requirements plan
Concept of operation
Concept of requirements
Prototype 1
Prototype 2
Operational prototype
Requirements
Draft
Detailed design
Development plan
Verification & Validation
Code
Integration
Test plan
Verification & Validation
Test
Implementation
4. Plan the next iteration
Release
3. Development and Test

Spiral

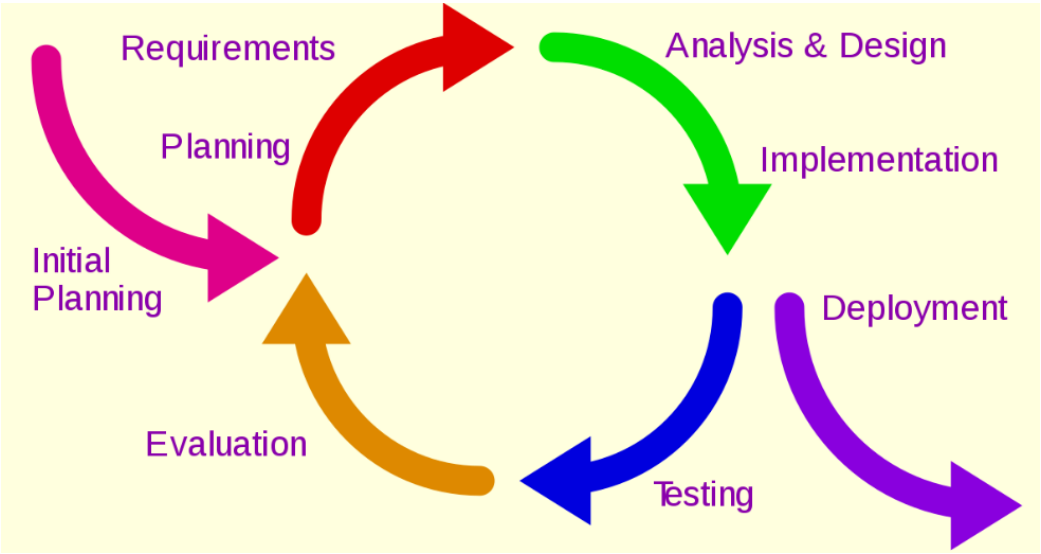
- Kulcs karakterisztika: kockázat menedzsment a fejlesztés megfelelő fázisában
- A projekt egyedi kockázatai alapján veszi át más folyamatmodellek elemeit (inkrementális, vízesés, evolúciós modellek)

Spiral

- Négy fő aktivitást ismétel ciklikusan (spirál formájában):
  1. Tervek kialakítása: célok, elvárások, megkötések
  2. Kockázatelemzés: kockázatot jelentő tényezők azonosítása és eltávolítása
  3. Implementáció: (alkalmazás)fejlesztés és verifikáció
  4. Eredmények kiértékelése és a következő iteráció tervezése

Iteratív és inkrementális

- Részegységekre osztjuk a feladatokat
- A rendszer ismétlődő ciklusok (iteratív) és egy időben kisebb részek kidolgozásával (inkrementálisan) fejlődik
  - Minden iterációban frissül a terv és új funkciók kerülnek kidolgozásra
  - Folyamatos finomítás alatt van a fejlesztés, egymással párhuzamosan is futnak a lépések



RUP módszertan

RUP

- IBM **R**ational **U**nified **P**rocess (RUP), 2003
- Átfogó folyamat keretrendszer
- Iparban alaposan tesztelt gyakorlatok szoftver és rendszer fejlesztésre, projektmenedzsmentre

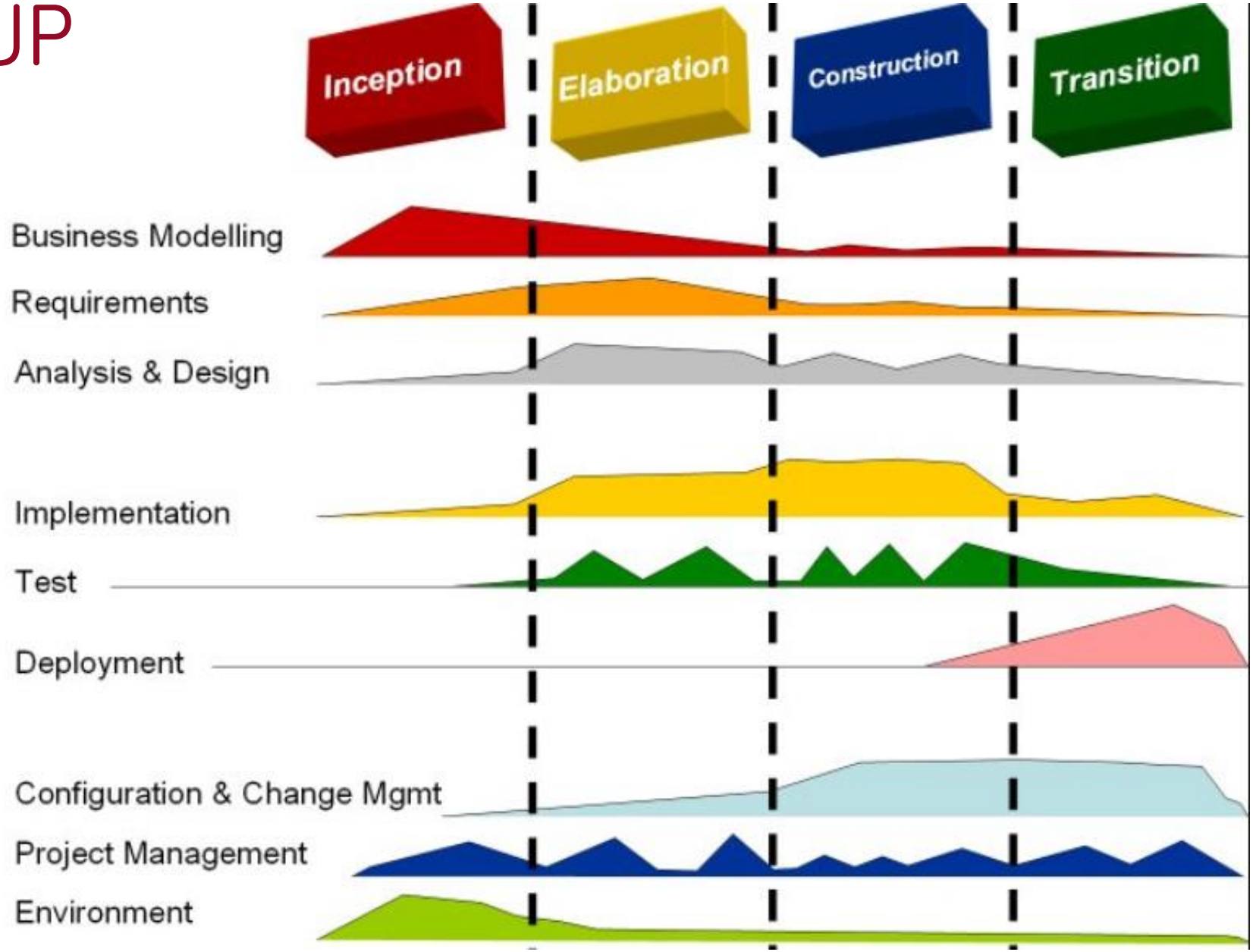
RUP

- ~ „best practice”-ek (legjobb gyakorlatok) átgondolt gyűjteménye
  - Több ezer projekt alapján
  - Használjuk a mások által sikerrel alkalmazott módszereket, folyamatokat
  - Projekt sablonok (felépítés, erőforrások, mérföldkövek, leszállítandók), dokumentumok

RUP

- Iteratív módszer
- Fázisokkal dolgozik

RUP


Inception
Elaboration
Construction
Transition
Business Modelling
Requirements
Analysis & Design
Implementation
Test
Deployment
Configuration & Change Mgmt
Project Management
Environment

RUP építőelemek

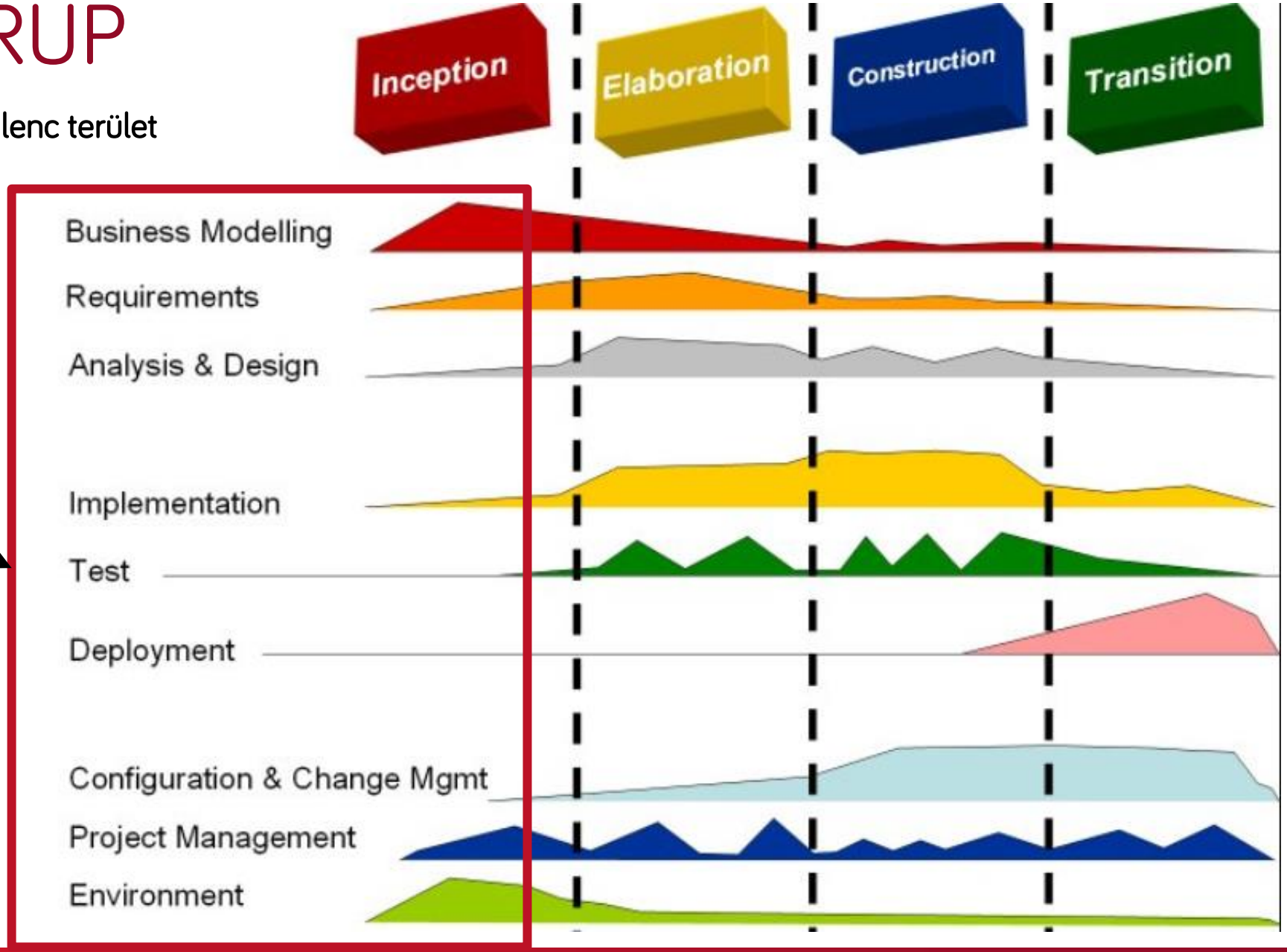
- Szerepkörök (who) – A szerepkör képességet, kompetenciát és felelősséget definiál.
- Eredménytermékek (what) – Az eredménytermék (Work Product) a feladatok eredményeit jelentik, beleértve a teljes folyamat során keletkezett modelleket és dokumentációt

RUP építőelemek

- Feladatok (how) – A feladat egy szerepkörhöz rendelt munkaegységet definiál, melynek használható eredménye van
- Minden iteráción belül a feladatok kilenc területre oszlanak fel:
- Hat „mérnöki” területre
  - Üzleti modellezés
  - Követelmények
  - Elemzés és tervezés
  - Implementáció
  - Tesztelés
  - Üzembe helyezés
- Három támogató területre
  - Konfiguráció és változáskövetés
  - Projektmenedzsment
  - Környezet

RUP

kilenc terület

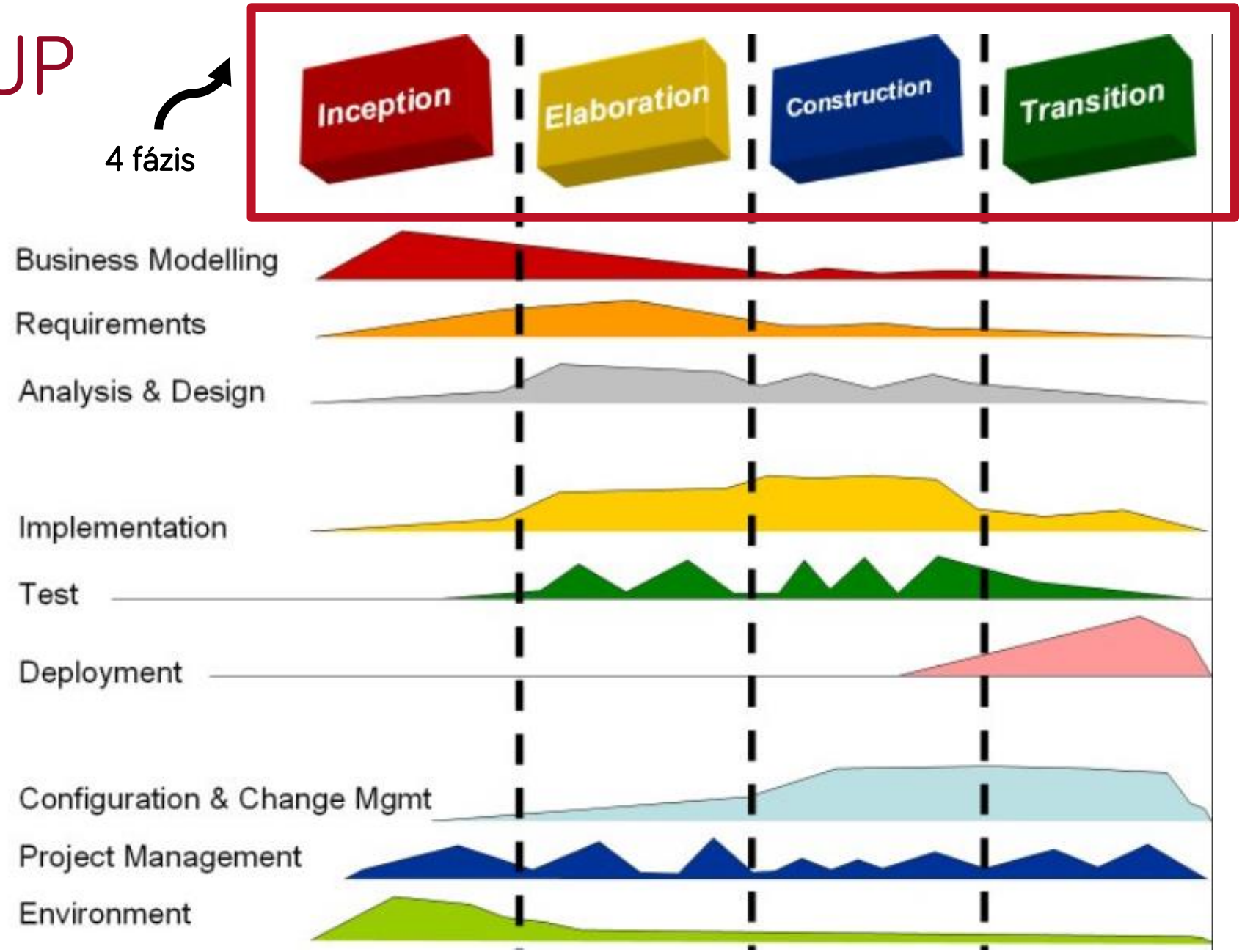


RUP – A projekt életciklus 4 fázisa

- Az iteratív fejlesztési folyamatot négy fázisra osztja
- Minden fázis egy vagy több iterációt tartalmaz
- Minden fázis alatt minden terület megjelenik, de más mértékben
- Minden teljes új ciklust új generációnak hívnak
  - Generációnkénti szerződés
  - Tipikusan néhány hónap hosszúságú

RUP

4 fázis


Inception
Elaboration
Construction
Transition
Business Modelling
Requirements
Analysis & Design
Implementation
Test
Deployment
Configuration & Change Mgmt
Project Management
Environment

RUP – Inception (Kezdet)

- A projektötlet kialakítása
- A csapat eldönti, hogy érdemes-e a projektet elindítani

Vagy

- A csapat eldönti, hogy érdemes-e a projektet érdemben folytatni
- A csapat megbecsüli, hogy a projektnek milyen erőforrás igénye lesz

RUP – Elaboration (Kidolgozás)

- A projekt architektúra és a szükséges erőforrásigény kidolgozása
- A fejlesztők átgondolják a lehetséges alkalmazási területeket, valamint a fejlesztés költségeit

RUP – Construction (Megépítés)

- A projekt ebben a szakaszban kidolgozásra kerül
- A hagyományos fázisok közül itt történik:
  - A szoftver tervezése
    - UML és egyéb mérnöki tervek, kevesebb szöveges dokumentáció
  - A szoftver implementációja/fejlesztése
  - A szoftver tesztelése

RUP – Transition (Átadás)

- A szoftver publikus kiadása, átadása
  - végső simítások
  - visszajelzések alapján történő korrekciók

A 6 best practice

1. Develop iteratively (Fejlesszünk iteratívan):
   - A legjobb minden követelményt előre ismerni, ugyanakkor ez gyakran nem így van.
2. Manage requirements (Kezeljük a követelményeket):
   - Mindig tartsuk szem előtt, hogy a követelményeket a felhasználók határozzák meg.

A 6 best practice

3. Use components (Használjunk komponenseket): A projekt komponensekre bontása segíti a tesztelést, újrafelhasználást, átlátást, menedzselést.
4. Model visually (Vizuálisan modellezünk): Használjunk diagramokat a fő komponensek, felhasználók, interakciók leírására (szoftver modellezés, UML, DSL).

A 6 best practice

5. Verify quality (Ellenőrizzük a minőséget):
   - A tesztelés mindig kapjon kitüntetett szerepet a projekt minden fázisában.
   - A tesztelési feladat folyamatosan nő a projekt előrehaladtával, de konstans faktornak kell lennie!

A 6 best practice

6. Control changes (Kövessük és ellenőrizzük a változásokat):
   - Számos projektet több különböző csapat fejleszt, gyakran különböző helyszínen, más-más platformok használatával.
   - Folyamatosan gondoskodjunk a változások szinkronizálásáról és verifikálásáról. (Continuous integration).

Köszönöm a figyelmet!